

Linux Kernel Internals: Memory Management, Process Hierarchies, and Control Interfaces

A Systems Implementation of Multi-Level Page Tables, Character Device Drivers, RCU Process Manipulation, and Kernel State Export

Pratik Wadekar

Department of Computer Science & Engineering
Systems Engineering Technical Report

Abstract

Operating system abstractions such as virtual memory, process trees, and kernel telemetry are ubiquitous in user-space software, yet their underlying mechanics are hidden beneath kernel system call interfaces. This report presents a comprehensive systems study and implementation of Linux kernel internals targeting the Linux 6.1.x kernel architecture. The project comprises five modular Linux Kernel Modules (LKMs), two custom `ioctl`-driven character device drivers, and two distinct kernel-to-user state export interfaces (`procfs` and `sysfs`). We investigate manual five-level page-table traversal (PGD to PTE) for virtual-to-physical address translation, maple-tree Virtual Memory Area (VMA) iteration for precise resident memory accounting, Transparent HugePage (THP) detection, live kernel process-tree surgery via Read-Copy-Update (RCU) linked-list manipulation, direct physical-memory write mechanisms, and `kobject`-based `sysfs` hierarchies. The report analyzes the structural trade-offs between asymptotic algorithmic complexity, locking synchronization guarantees, and hardware memory safety.

1. INTRODUCTION AND SYSTEMS MOTIVATION

The Linux kernel enforces a strict privilege boundary between user space and kernel space. User-space utilities such as `ps`, `top`, and `vmstat` rely on synthetic filesystems (`/proc` and `/sys`) to query process states, memory consumption, and hardware statistics. Similarly, memory management primitives (`mmap`, `brk`) provide processes with virtual address spaces that are dynamically translated and backed by physical pages via hardware-assisted multi-level page tables.

The primary objective of this project is to develop and analyze custom Linux Kernel Modules that interact directly with core kernel data structures:

- **Virtual Memory Introspection:** Manually traversing page table hierarchies down to individual Page Table Entries (PTEs) without relying on high-level kernel helpers.
- **Memory Accounting Mechanics:** Comparing $O(1)$ approximate Resident Set Size (RSS) metrics against $O(\text{pages})$ exact VMA scans using modern Maple Tree iterators (`VMA_ITERATOR`).
- **Kernel Control Channels:** Building character device drivers exposing custom `ioctl` interfaces to perform physical memory writes and live process tree mutations.
- **Kernel State Telemetry:** Exposing live kernel metrics through both sequential single-counter `procfs` endpoints and multi-attribute `sysfs` `kobject` hierarchies.

2. VIRTUAL MEMORY ARCHITECTURE AND MANUAL PAGE TABLE WALKS

In modern 64-bit architectures (x86_64 and arm64), virtual memory translation is implemented via a multi-level radix-tree hierarchy managed by hardware Memory Management Units (MMUs) and the Linux memory subsystem.

2.1 Five-Level Page Table Traversal Mechanics

Rather than using opaque kernel wrappers (such as `get_user_pages()`), the diagnostic modules (`lkm3.c`, `lkm4.c`, `lkm5.c`, `chardev.c`) manually resolve virtual addresses through a 5-level walk:

```
unsigned long page_table_walk(struct mm_struct *mm, unsigned long vaddr) {
    pgd_t *pgd; p4d_t *p4d; pud_t *pud; pmd_t *pmd; pte_t *pte;
    pgd = pgd_offset(mm, vaddr);
    if (pgd_none(*pgd) || pgd_bad(*pgd)) return 0;
    p4d = p4d_offset(pgd, vaddr);
    if (p4d_none(*p4d) || p4d_bad(*p4d)) return 0;
    pud = pud_offset(p4d, vaddr);
    if (pud_none(*pud) || pud_bad(*pud)) return 0;
    pmd = pmd_offset(pud, vaddr);
    if (pmd_none(*pmd) || pmd_bad(*pmd)) return 0;
    pte = pte_offset_kernel(pmd, vaddr);
    if (!pte || !pte_present(*pte)) return 0;
    return (pte_pfn(*pte) << PAGE_SHIFT) | (vaddr & ~PAGE_MASK);
}
```

At each level, validation macros guard against invalid or uncommitted table entries:

1. **Presence and Badness Checks:** `pgd_none()` and `pgd_bad()` verify entry validity before calculating sub-table pointers.
2. **Physical Frame Extraction:** The terminal entry yields a Page Frame Number (PFN) via `pte_pfn(*pte)`, which is shifted by `PAGE_SHIFT` (12 bits for 4KB pages) and masked with the in-page offset (`vaddr & ~PAGE_MASK`).
3. **Architecture Portability:** Using `pte_pfn()` provides architectural portability across both x86_64 and arm64 platforms without relying on architecture-specific bitmasks (`PTE_PFN_MASK` vs. `PTE_ADDR_MASK`).

```

Linux Kernel Internals Architecture (Target: Linux Kernel 6.1.x):
+-----+
| Part 1: Modular Kernel Diagnostics (1/) |
| - lkm1: Iterates task_struct list via for_each_process(), filters runnable tasks |
| - lkm2: RCU child traversal via list_for_each_entry_rcu() with state decoding |
| - lkm3: Manual 5-level page-table walk (PGD -> P4D -> PUD -> PMD -> PTE) for VA->PA |
| - lkm4: Dual memory sizing: O(1) fast get_mm_rss() vs O(pages) precise Maple Tree VMA walk |
| - lkm5: Transparent HugePage (THP) accounting via PMD-level pmd_trans_huge() checks |
+-----+
| Part 2: Character Device Control Drivers (2.1/, 2.2/) |
| - chardev 2.1: unlocked_ioctl driver for VA->PA query and direct phys_to_virt() write |
| - chardev 2.2: Live process reparenting via list_del_init/list_add_tail_rcu and mass SIGTERM |
+-----+
| Part 3: Kernel Telemetry Export Subsystems (3.1/, 3.2/) |
| - procfs (3.1): Single-value /proc/get_pgfaults endpoint using proc_ops and all_vm_events() |
| - sysfs (3.2): kobject hierarchy at /sys/kernel/mem_stats/{pid, virtmem, physmem, unit} |
+-----+

```

Figure 1: System Overview: Diagnostic LKMs, IOCTL Device Drivers, and Procfs/Sysfs Interfaces

```

Live Process Hierarchy Reparenting Mechanics (change_parent):
Initial State (Shell as Parent):
[Shell (PID 100)] ----> children list ----> [Soldier Task (PID 200)] (parent = 100)
[Control Station (PID 150)] -> children list -> []

Kernel List Surgery (IOCTL_SEND_SIGCHLD_TO_PID called by Soldier):
1. list_del_init(&current->sibling); // Remove Soldier from Shell's children
2. list_add_tail_rcu(&current->sibling, &cs->children); // Splice into Control Station list
3. RCU_INIT_POINTER(current->parent, cs_task); // Overwrite parent pointers
4. RCU_INIT_POINTER(current->real_parent, cs_task);

Resulting State (Control Station as Adoptive Parent):
[Control Station (PID 150)] ----> children list ----> [Soldier Task (PID 200)] (parent = 150)
* When Soldier terminates, SIGCHLD is delivered to Control Station; waitpid() reaps child.

```

Figure 2: Process Hierarchy Splicing: Decoupling Birth Lineage from Runtime Signal Delivery

2.2 Process Memory Accounting: Fast Approximate vs. Exact VMA Walks

Process memory sizing exhibits a fundamental engineering trade-off between execution speed and accounting precision (lkm4.c):

- **Virtual Address Space Sizing:** Evaluated in $O(1)$ time by reading `mm->total_vm << PAGE_SHIFT`, reflecting the cumulative span of allocated VMAs regardless of physical page allocation.
- **Fast Approximate RSS:** Evaluated in $O(1)$ time via `get_mm_rss(mm)`, querying internal atomic counters updated during page fault allocations and unmappings (the source used by `/proc/<pid>/statm`).
- **Exact Present Page Accounting:** Evaluated in $O(\text{pages})$ time by traversing every VMA using the Linux 6.1 Maple Tree API (`VMA_ITERATOR(vmi, mm, 0)` and `for_each_vma(vmi, vma)`). The kernel iterates across each 4 KB stride within `[vma->vm_start, vma->vm_end)`, executing the 5-level walk to count only physically mapped frames.

2.3 Transparent HugePage (THP) Traversal Mechanics

To account for Transparent HugePages (2 MB contiguous physical pages on x86_64), the memory scanner (lkm5.c) inspects entries at the PMD level:

```

if (pmd_trans_huge(*pmd)) {
    stats->thp_count++;
    stats->thp_size += HPAGE_PMD_SIZE;
    vaddr += HPAGE_PMD_SIZE - PAGE_SIZE; // Advance cursor by full huge-page stride
}

```

Detecting huge pages via `pmd_trans_huge()` allows the traversal loop to advance by `HPAGE_PMD_SIZE` in a single step, skipping 512 constituent 4 KB leaf checks and preventing double-counting.

3. CHARACTER DEVICE DRIVERS AND CUSTOM IOCTL CONTROL CHANNELS

To bridge user-space applications with kernel mechanics, the project implements two character device drivers registered at `/dev/chardev` using `alloc_chrdev_region`, `cdev_init`, `class_create`, and `device_create`.

3.1 Subsystem 2.1: Address Translation and Physical Memory Writes

The first driver (2.1/chardev.c) exposes two structured commands defined in a shared ABI header (chardev.h):

- **IOCTL_PHYS_TO_VIRT:** Takes a user virtual address via `struct query_arg_t`, executes `page_table_walk(current->mm, vaddr)`, and returns the resolved physical address across the user-kernel boundary via `copy_to_user()`.
- **IOCTL_WRITE_TO_PHYS:** Accepts a physical address and a byte payload, writing directly to physical memory using the kernel direct-mapping macro `*(char *)phys_to_virt(phys_addr) = byte_val`. The companion user client (dev.user.c) verifies that writing to the physical address immediately updates the process's own virtual memory buffer without calling user-space write routines.

3.2 Subsystem 2.2: Live Process Tree Surgery and Coordinated Termination

The second driver (2.2/chardev.c) demonstrates direct kernel state mutation by manipulating the operating system's process tree:

1. **Dynamic Process Reparenting (change_parent):** Modifies `current->real_parent` and `current->parent` pointers under `rcu_read_lock()`. The calling task is unlinked from its original parent via `list_del_init(¤t->sibling)` and spliced onto the target parent list via `list_add_tail_rcu()`. Consequently, the target process (`control_station.c`) receives `SIGCHLD` and reaps the adopted child via non-blocking `waitpid(-1, &status, WNOHANG)` loops despite never having invoked `fork()`.
2. **Coordinated Mass Termination (kill_all):** Traverses modified children via `list_for_each_entry_rcu()`, dispatches `send_sig(SIGTERM, child, 1)` to each child process, pauses for a 2-second grace period via `msleep(2000)`, and dispatches `SIGTERM` to the parent task.

4. KERNEL STATE TELEMETRY: PROCFS AND SYSFS SUBSYSTEMS

4.1 Single-Counter Procfs Interface (3.1/get_pgfaults.c)

The procfs interface registers `/proc/get_pgfaults` using `proc_create()` and `struct proc_ops`:

```
static ssize_t procfile_read(struct file *file, char __user *buf, size_t count, loff_t *pos) {
    unsigned long events[NR_VM_EVENT_ITEMS];
    all_vm_events(events); // Snapshot global per-CPU VM statistics
    len = sprintf(page_faults, "%lu\n", events[PGFAULT]);
    return simple_read_from_buffer(buf, count, pos, page_faults, len);
}
```

Reading `/proc/get_pgfaults` aggregates per-CPU page-fault event counters dynamically, exposing identical telemetry to the system-wide `/proc/vmstat` endpoint.

4.2 Structured Sysfs and Kobject Hierarchy (3.2/get_memstats.c)

The sysfs implementation constructs a directory at `/sys/kernel/mem_stats/` by registering a custom kobject with `kernel_kobj`:

- **Intrusive Struct Embedding:** A custom state structure (`struct sysfs_entry`) encapsulates the base kobject, target process PID, and unit configuration ('B', 'K', 'M'). Sysfs attribute handlers retrieve instance state via `container_of(kobj, struct sysfs_entry, kobj)`.
- **Dynamic Attribute Handlers:** Exposes four attributes via `ATTRIBUTE_GROUPS`:
 - `pid` (read/write): Validates input strings via `kstrtoint()`, queries `find_vpid(pid)`, and stores active PIDs.
 - `virtmem` (read-only): Resolves the target `task_struct`, reads virtual memory span, and formats output.
 - `physmem` (read-only): Executes a full per-VMA page table walk over the target process's memory space, reporting precise resident memory in the selected unit.
 - `unit` (read/write): Configures output scaling factors between Bytes, Kilobytes, and Megabytes.

5. SYSTEMS ENGINEERING, CONCURRENCY, AND KERNEL SUBTLETIES

5.1 Locking Invariants in Process Hierarchy Manipulation

In `2.2/chardev.c`, `change_parent()` mutates process list pointers within an `rcu_read_lock()` section. In the Linux kernel core (`kernel/exit.c`), process reparenting requires acquiring the write lock on the tasklist lock:

```
write_lock_irq(&tasklist_lock);
// Mutate parent and children list heads safely against concurrent schedulers
write_unlock_irq(&tasklist_lock);
```

Because `tasklist_lock` is not exported to out-of-tree loadable kernel modules, module-level reparenting relies on RCU synchronization for list insertions. While sufficient for sequential demonstrative flows, concurrent unlinking under high scheduling contention requires formal spinlock serialization to prevent list corruption.

5.2 Direct-Mapping vs. High-Memory Physical Access

The physical write handler in `2.1/chardev.c` converts physical addresses using `phys_to_virt(phys_addr)`. This operation is valid for memory located within the kernel's direct identity-mapping region (low memory). For high-memory regions on 32-bit systems or non-identity-mapped physical frames, general-purpose drivers require temporary architectural mappings using `kmap_local_pfn()` and `kunmap_local()` to map physical frames into kernel address space safely.

5.3 Structure Alignment and IOCTL Command Encoding

In `2.1/chardev.h`, `_IOWR` and `_IOW` command macros encode direction, magic numbers, sequence indices, and type sizes. Ensuring exact struct alignment across user-space and kernel compilers prevents data corruption across 32-bit and 64-bit boundaries. Explicit `copy_from_user()` and `copy_to_user()` invocations protect kernel stack memory from arbitrary unvalidated user pointers.

6. DESIGN TRADE-OFFS AND ARCHITECTURAL OPTIMIZATIONS

- **Manual Page-Table Walk vs. Kernel Helpers:** Implementing explicit five-level traversals demonstrates the MMU translation process directly. However, in production drivers, utilizing `get_user_pages()` or `follow_pfn()` automatically handles page faulting, copy-on-write resolution, and locking of the target `mm->mmap_lock`.
- **Fast Approximations vs. Complete VMA Iteration:** $O(1)$ counter reads (`get_mm_rss`) provide immediate telemetry with zero traversal overhead, whereas $O(\text{pages})$ walks guarantee exact physical residency at the expense of higher CPU cycle consumption on multi-gigabyte address spaces.
- **IOCTL Request-Response vs. File System Telemetry:** Binary `ioctl` control channels provide low-overhead, structured, two-way communication for complex memory queries, while `procfs` and `sysfs` offer human-readable, discoverable text

interfaces integrated with standard VFS tooling.

- **Memory Synchronization during VMA Iteration:** In high-concurrency environments, acquiring read locks on the target `mmap_lock` prevents race conditions against concurrent `mmap` or `munmap` calls during per-page address translation walks.

7. REPRODUCTION AND EXECUTION GUIDE

The codebase is structured into self-contained subdirectories, each providing standalone Makefile and Kbuild configurations targeting Linux 6.1.x:

- **Part 1 Diagnostic Modules:**

```
cd 1/ && make
sudo insmod lkm3.ko pid=<PID> vaddr=<VADDR> && dmesg | tail
sudo insmod lkm4.ko pid=<PID> && dmesg | tail
sudo rmmod lkm3 lkm4
```

- **Part 2 IOCTL Character Device Drivers:**

```
cd 2.1/ && make && sudo insmod chardev.ko && sudo ./dev_user 3 && sudo rmmod chardev
cd 2.2/ && bash run_dr_bloom.sh
```

- **Part 3 Procfs and Sysfs Interfaces:**

```
cd 3.1/ && make && sudo insmod get_pgfaults.ko && cat /proc/get_pgfaults && sudo rmmod get_pgfaults
cd 3.2/ && make && sudo insmod get_memstats.ko
echo <PID> > /sys/kernel/mem_stats/pid && cat /sys/kernel/mem_stats/physmem
sudo rmmod get_memstats
```

8. CONCLUSION

This project demonstrates the core mechanisms governing Linux kernel memory management, process hierarchy scheduling, and user-kernel communication interfaces. By implementing manual 5-level page table walks, Maple Tree VMA iteration, Transparent HugePage accounting, RCU-based process tree surgery, direct physical memory access, and sysfs kobject hierarchies, the work illuminates how foundational operating system abstractions are realized within the Linux kernel architecture.